LIBXSTREAM Samples

Ozaki Scheme - OpenCL

This sample demonstrates high-precision GEMM emulation via **mantissa slicing** (Ozaki Scheme 1) and **Chinese Remainder Theorem** (Ozaki Scheme 2), both fully offloaded to an OpenCL device. It is an OpenCL adaptation of the CPU-based Ozaki sample in LIBXS.

Algorithm

Scheme 1 - Mantissa Slicing Two kernel variants implement the mantissa decomposition:

- **int8** (`ozaki1_int8.cl`, default, `OZAKI=1`) - Each IEEE-754 mantissa is decomposed into 7-bit signed int8 slices with a shared per-row/per-column exponent. The DPAS path uses `i8_i8_matrix_mad_k32` (BK=32).

The GEMM `C = alpha * A * B + beta * C` is then computed as a sum of `S*(S+1)/2` (with triangular + symmetrize) pairwise dot products over `BMxBNxBK` tiles, matching the granularity of GPU matrix engines.

Scheme 2 - Chinese Remainder Theorem (CRT)

- **u8 default** (`ozaki2_int8.cl`, `OZAKI=2`) - Each IEEE-754 mantissa, shifted by a per-row/per-column max exponent, is reduced modulo each of up to 20 pairwise coprime moduli. One dot product is performed per modulus channel (NPRIMES independent products instead of S*(S+1)/2 pairwise slice products). The results are reconstructed via **Garner's algorithm** and evaluated with **Horner's method** in a mixed-radix representation.

  Residues are unsigned uint8 with moduli up to 256 (prime powers $256=2^{8,}$ $^{243=3}5$, 169=13^2 alongside large primes). Sign is encoded via the modular additive inverse (p - r). The DPAS path uses `u8_u8_matrix_mad_k32`; a signed i8 equivalent is available via `OZAKI_I8=1`.

  XMX / DPAS acceleration uses a **fused** `dotprod` kernel that computes DPAS matmuls per modulus channel, reduces to unsigned residues in registers, and immediately runs Garner reconstruction and Horner evaluation - no intermediate global buffer is needed. A scalar fallback follows the same fused structure. The triangular and symmetrize optimisations do not apply (each modulus channel is independent).

  K-grouping (`OZAKI_GROUPS`): When `OZAKI_GROUPS` > 1, that many consecutive K sub-panels share a common max exponent and their DPAS dot products are accumulated before a single Garner reconstruction per group. This amortises the (expensive) Garner/Horner phase across OZAKI_GROUPS panels. With `OZAKI_GROUPS=4` the Garner cost is cut by 4x and overall speedups of ~3x have been measured on PVC at large matrix sizes.

Pipeline Both schemes share the same three-kernel pipeline:

1. **preprocess_a** - decompose rows of A into int8 slices (Scheme 1) or modular residues (Scheme 2)
2. **preprocess_b** - decompose columns of B into int8 slices or residues
3. **dotprod** - iterate slice pairs or modulus channels, accumulate into C

Scheme 2 fuses Garner reconstruction and Horner evaluation directly into the `dotprod` kernel (both XMX and scalar paths), eliminating the need for an intermediate residue buffer or a separate postprocess phase.

Shared IEEE-754 field extraction is factored into `ozaki_common.cl` (`ieee_decompose`), included by both `ozaki1_int8.cl` and `ozaki2_int8.cl`. Scheme 2 additionally factors Garner reconstruction and Horner evaluation into file-local inline helpers (`oz2_garner_reconstruct`, `oz2_horner_accumulate`) shared between the XMX and scalar `dotprod` paths.

When Intel XMX hardware is detected (`cl_intel_subgroup_matrix_multiply_accumulate` and `cl_intel_subgroup_2d_block_io`), the Scheme 1 dotprod kernel uses **DPAS** (Data Processing Accelerator Systolic) instructions with 2D block I/O for data movement (SG=16, 8x32 A reads, 32x16 VNNI-transformed B reads). Otherwise a scalar fallback is used.

A double-buffered K-batch pipeline overlaps preprocessing of batch N+1 with the dot-product computation of batch N across three concurrent streams.

TinyTC Kernels Optionally, Scheme 1 can use **TinyTC**-generated SPIR-V kernels for the dotprod phase. TinyTC is a tensor compiler that generates optimized GPU code. Precompiled `.clx` files in the `kernels/` directory can be loaded at runtime (via `OZAKI_TINYTC` env var) or embedded at build time (with `-DOZAKI_TINYTC_EMBED`). The filenames encode block sizes and configuration: `ozaki1_{f64|f32}_{BM}x{BN}_n{N}t{TRIM}_{tri|sq}.clx`.

Performance Features **Device Memory Pool**: When Intel USM or OpenCL SVM is available, `ozaki_init` creates a device-memory pool (backed by `libxs_malloc_xpool` with `LIBXS_MALLOC_NATIVE`). Buffers allocated during `ozaki_gemm` are returned to the pool instead of being freed, eliminating per-call allocation overhead for repeated GEMM calls. With the pool active, `ozaki_gemm` returns without synchronizing - the caller is responsible for syncing the stream. On the rare grow path (larger problem size), the wrapped deallocator syncs all streams before reallocating. The pool falls back to direct `libxstream_mem_allocate` transparently if USM/SVM is not supported.

Preprocessing Cache: When `OZAKI_CACHE` is enabled (bitmask: 1=A, 2=B, 3=both), preprocessed slices and exponents are cached per input matrix pointer. If the same matrix is passed to `ozaki_gemm` with matching dimensions and transpose flag, preprocessing is skipped entirely. This is particularly effective when A or B remains constant across multiple calls (e.g., batched inference with a fixed weight matrix).

Build

```
cd samples/ozaki
make [GNU=1] [DBG=1] [PEDANTIC=2]
```

Requires an OpenCL runtime and headers (e.g., `opencl-c-headers`, `ocl-icd-dev`, or Intel oneAPI). BLAS is linked via `BLAS=2` (2=parallelized) for the reference GEMM.

Run

```
./ozaki.x [M [N [K [transa [transb [alpha [beta [lda [ldb [ldc]]]]]]]]]]
```

All arguments are positional and optional (defaults shown):

Position	Argument	Default	Description
1	`M`	257	Number of rows of C and op(A)
2	`N`	M	Number of columns of C and op(B)
3	`K`	M	Inner dimension
4	`transa`	0	0 = 'N', 1 = 'T' for A
5	`transb`	0	0 = 'N', 1 = 'T' for B
6	`alpha`	1	Scalar multiplier for A*B
7	`beta`	1	Scalar multiplier for C
8	`lda`	auto	Leading dimension of A
9	`ldb`	auto	Leading dimension of B
10	`ldc`	M	Leading dimension of C

Environment Variables

Variable	Default	Description
`OZAKI`	1	Kernel variant for real GEMM: 1 = int8 mantissa slices, 2 = int8 CRT.
`OZAKI_3M`	*auto*	Complex GEMM dispatch (ZGEMM/CGEMM): 0 = original BLAS, 1 = CPU 3M, 2 = GPU 3M (falls back to CPU). Default: 0 if `OZAKI=0`, else 2. Controlled by LIBXS.
`OZAKI_MAXK`	32768	Max K elements per preprocessing pass. 0 = no grouping (full K in one pass). Trades exponent-scope precision vs FP accumulation rounding. Applies to both CPU and GPU.
`OZAKI_FLAGS`	3	Scheme 1 bitmask: Triangular (1), Symmetrize (2). 0 = full S^2 square. Ignored for Scheme 2.
`OZAKI_TRIM`	0	Precision levels to trim. Scheme 1: drops diagonals from slice-pair iteration ($_{7}$ product bits/level). Scheme 2: truncates mantissa before CRT ($_{2}$ input bits/level, ~4 product bits/level). The level semantics are calibrated so the same trim value gives comparable accuracy across schemes.
`OZAKI_N`	8/16	Number of decomposition components. Scheme 1: number of slices per element (fp64: default 8, fp32: default 4). Scheme 2: number of CRT primes (fp64: default 16, fp32: default 9; max 20).
`OZAKI_I8`	0	Scheme 2 only: use signed i8 residues (moduli <= 128) instead of the default unsigned u8.
`OZAKI_GROUPS`	0	Scheme 2 only: K-grouping factor - that many consecutive K sub-panels share one exponent and one Garner reconstruction (0/1 = no grouping, 4 = quads).
`OZAKI_CACHE`	0	Preprocessing cache bitmask: 1 = cache A, 2 = cache B, 3 = cache both. Skips preprocessing when same matrix pointer is reused with matching dims/transpose.
`OZAKI_TINYTC`	0	Load TinyTC SPIR-V kernel from .clx file path (e.g., `kernels/ozaki1_f64_256x128_n8t3_tri.clx`). 0 = use embedded or OpenCL C kernels.
`OZAKI_VERBOSE`	0	Verbosity level: 0 = silent, 1 = errors only, 2 = errors + warnings, 3+ = all info. Negative values also enable all output.
`OZAKI_XMX`	auto	Override XMX detection (0 = force off, 1 = on).
`OZAKI_WG`	0	Work-group size hint (0 = no hint).
`OZAKI_SG`	auto	Sub-group size (forced to 16 when XMX active).
`OZAKI_BIGGRF`	auto	Override 256 GRF per thread (0 = force off, 1 = on). Auto-enabled for Intel GPUs.
`OZAKI_FP`	64	Floating-point precision: 64 = fp64 (double), 32 = fp32 (float).
`OZAKI_RTM`	auto	Register tiling factor for M

dimension (power of two). Auto: 4 (256-GRF Intel GPU), 2 (128-GRF), 1 (other). | | `OZAKI_RTN` | auto | Register tiling factor for N dimension (power of two). Auto: 2 (Intel GPU), 1 (other). | | `OZAKI_KU` | 2 | K-loop unroll factor (compiled into kernel). | | `OZAKI_RC` | 8 | DPAS repeat count: 8 (default) or 4 (split). | | `OZAKI_PB` | 1 | Scheme 2 only: CRT prime batching factor (compiled into kernel). | | `OZAKI_PREFETCH` | 0 | 1 = enable prefetching in Scheme 1 dotprod kernel. | | `OZAKI_BOUNDS` | 0 | 1 = force bounds checking in Scheme 1 dotprod kernel (automatic for non-tile-aligned sizes). | | `OZAKI_SCALAR_ACC` | 0 | 1 = force scalar accumulation in Scheme 1 dotprod kernel. | | `NREPEAT` | 1 | Number of times to repeat the benchmark (for timing measurements). |

Additional environment variables (`OZAKI_PROFILE`, `OZAKI_THRESHOLD`, `OZAKI_STAT`, `OZAKI_EPS`, `OZAKI_RSQ`, `OZAKI_EXIT`) are handled by the LIBXS Ozaki sample which owns the GEMM interceptor, histogram-based profiling, and accuracy monitoring. See the LIBXS Ozaki README for their documentation.

The Ozaki context auto-selects XMX-friendly defaults when hardware support is detected. For int8 Scheme 1 (default): `BK=32`, `BM=16`, `BN=16`. For CRT Scheme 2: XMX uses `BK=32` with fused in-register Garner/Horner. Common defaults: `SG=16`. Scheme 2 auto-default for fp64 is 16 primes (u8) or 19 primes (i8 with `OZAKI_I8=1`).

Example

```
$ ./ozaki.x 256
OpenCL benchmark for Ozaki's methods
GEMM: NN M=256 N=256 K=256 lda=256 ldb=256 ldc=256 alpha=1 beta=1
Ozaki GEMM: 12.345 ms
BLAS  GEMM: 1.234 ms
DIFF: linf=0.000000 linf_rel=0.000000 l2_rel=0.000000 eps=0.000000 rsq=1.000000
```

With Scheme 2 CRT and K-grouping:

```
$ OZAKI=2 OZAKI_GROUPS=4 ./ozaki.x 4096
OpenCL benchmark for Ozaki's methods
GEMM: NN M=4096 N=4096 K=4096 lda=4096 ldb=4096 ldc=4096 alpha=1 beta=1
Device: Intel(R) Data Center GPU Max 1550 (GPU)
Ozaki GEMM: 234.567 ms
BLAS  GEMM: 12.345 ms
DIFF: linf=0.000000 linf_rel=0.000000 l2_rel=0.000000 eps=0.000000 rsq=1.000000
```

Files and Directory Structure

| File/Directory | Description | ||| | `ozaki_bench.c` | Main benchmark driver (initializes context, runs GEMM, compares with BLAS) | | `ozaki_gemm.c` | GEMM implementation (preprocessing pipeline, dotprod launch, buffer management) | | `ozaki_opencl.c` | Context initialization (device selection, kernel compilation, parameter tuning) | | `ozaki_gemm3m.c` | Complex GEMM via 3M (Karatsuba) method - all intermediates stay on device | | `ozaki_opencl.h` | Public API and context structure definitions | | `ozaki_kernels.h` | Auto-generated header embedding OpenCL C kernel sources | | `ozaki_tinytc.h` | Auto-generated header embedding specialized TinyTC SPIR-V kernels | | `kernels/ozaki1_int8.cl` | Scheme 1 OpenCL C kernels (preprocess + XMX/scalar dotprod) | | `kernels/ozaki2_int8.cl` | Scheme 2 OpenCL C kernels (preprocess + fused CRT dotprod) | | `kernels/gemm3m.cl` | Complex GEMM 3M helper kernels (deinterleave, matadd, finalize) | | `kernels/ozaki_common.cl` | Shared IEEE-754 field extraction helpers | | `kernels/ozaki1.tinytc` | TinyTC kernel definition for generic Scheme 1 dotprod | | `kernels/ozaki1_prod_*.tinytc` | Specialized TinyTC definitions (triangular/square) | | `kernels/*.clx` | Precompiled TinyTC SPIR-V binaries (embedded at build time or loaded at runtime) |

Performance Tuning

Scheme Selection

- **Scheme 1 (int8 mantissa slicing)**: Better suited for smaller matrices and when high accuracy is needed with moderate slice counts (default: 8 slices for fp64). Triangular + symmetrize optimizations reduce slice-pair count from S^2 to S(S+1)/2. Use `OZAKI_TRIM` to drop least significant diagonals for speed at the cost of accuracy.
- **Scheme 2 (CRT)**: More efficient for large matrices, especially with K-grouping. Default 18 primes provide excellent accuracy. Use `OZAKI_GROUPS=4` on large GEMMs to amortize Garner reconstruction cost across K panels - speedups of 3x have been observed on Intel Data Center GPU Max (PVC) at sizes >=2048.

Cache and Memory

- Enable `OZAKI_CACHE=3` when repeatedly calling GEMM with the same A and B matrices (e.g., batched inference with fixed weights). Preprocessing is skipped entirely on cache hits, yielding 2-5x speedup for the overall GEMM.
- The device memory pool (requires USM/SVM) eliminates allocation overhead across repeated calls (`export OZAKI_DEVPOOL=1`). If your workload involves many different matrix sizes, consider keeping it disabled.

TinyTC vs OpenCL C

- TinyTC kernels are specialized at compile time per (ndecomp, trim, scheme) and offer 10-30% better performance than the generic OpenCL C kernels for Scheme 1. Use `OZAKI_TINYTC=kernels/ozaki1_f64_256x128_n8t3_tri.clx` to load a specific variant, or embed them at build time with `-DOZAKI_TINYTC_EMBED` (requires `.clx` files in `kernels/`).
- The OpenCL C kernels are always available as fallback and support runtime parameterization. TinyTC support is Scheme 1 only.

Profiling Profiling is controlled by the LIBXS Ozaki sample via the `OZAKI_PROFILE` environment variable. Both GPU and CPU paths push into the same histogram, and at exit a summary line reports effective GFLOPS/s together with the implied INT8-TOPS/s (derived from the scheme's decomposition multiplier). See the LIBXS Ozaki README for details on profile modes and output format.

Limitations

- Complex GEMM (3M method) is implemented (`ozaki_gemm3m` in `ozaki_gemm3m.c`) but not called from this sample's benchmark driver. It is exercised by the LIBXS Ozaki sample, whose interceptor-based driver dispatches ZGEMM/CGEMM through the GPU 3M path when LIBXSTREAM is available. The `OZAKI_3M` environment variable (0 = original BLAS, 1 = CPU 3M, 2 = GPU 3M) controls the complex GEMM dispatch; see the LIBXS Ozaki README for details.
- TinyTC kernels are Scheme 1 only (Scheme 2 uses OpenCL C kernels).

Small Matrix Multiplication (SMM) - OpenCL

This sample implements the ACC LIBSMM interface for batched small matrix multiplications (SMM) on OpenCL devices. It originates from DBCSR's OpenCL backend and has been adapted to run on top of LIBXSTREAM. The implementation includes GPU-accelerated kernels for batched matrix multiplication and transpose, a benchmark driver (`acc_bench`), and a complete auto-tuning framework.

Build

```
cd samples/smm
make [WITH_GPU=<device>] [ELEM_TYPE=<type>]
```

Produces `acc_bench.x`. Requires an OpenCL runtime, BLAS (linked via `BLAS=2`), and LIBXS built from a sibling directory.

Make variable	Default	Description
`ELEM_TYPE`	`double`	Element precision: `double` or `float`.
`WITH_GPU`	auto	Device name for selecting tuned parameters, e.g., `PVC`, `A100`, `V100`. Falls back to all CSV files in `params/`.

Benchmark Driver

```
./acc_bench.x [nrepeat [batchsize [M [N [K [nc [na [nb]]]]]]]]
```

The kernel shape can also be given as `MxNxK`:

```
./acc_bench.x 5 30000 13x5x7
```

Alternatively, the first argument can be a file containing one set of parameters per line (for batch testing multiple kernels).

| Argument | Default | Description | |||| | `nrepeat` | 66 (or 3 with `CHECK`) | Number of timed repetitions | | `batchsize` | 30000 | Number of matrix products per batch (the "stack size") | | `M` | 23 | Rows of A and C | | `N` | M | Columns of B and C | | `K` | M | Columns of A / rows of B | | `nc` | batchsize/16 | Number of unique C-matrices | | `na` | 10*nc* / *Number of unique A-matrices* | | *`nb`* | *10*nc | Number of unique B-matrices |

The batchsize argument also accepts memory-budget notation with K/M/G suffixes (e.g., `256M`), in which case the stack size is derived from the memory budget and matrix dimensions.

Environment Variables (Benchmark)

| Variable | Default | Description | |||| | `CHECK` | -1 (enabled) | Accuracy validation: negative = auto-threshold, 0 = disabled, positive = custom threshold. | | `CHECK_H2D` | - | Minimum H2D bandwidth (GB/s); fail if below. | | `CHECK_DEV` | - | Minimum device GFLOPS/s; fail if below. | | `CHECK_HST` | - | Minimum host GFLOPS/s; fail if below. | | `DEVICE` | 0 | Device index (with multiple ranks, device = rank % ndevices). | | `NREPEAT_H2D` | 1 | Number of H2D copy repetitions for bandwidth measurement. | | `NREPEAT_SMM` | 1 | Number of SMM kernel launches per timed iteration (for profiling). | | `BATCHSIZE_SMM` | - | Override batchsize and optionally nrepeat (`batchsize,nrepeat`). |

Compile-Time Knobs (Benchmark)

| Macro | Default | Description | |||| | `ELEM_TYPE` | `double` | Element type (`double` or `float`). | | `BATCHSIZE` | 30000 | Default batch/stack size. | | `BATCHGRAIN` | 100 | Granularity for randomized batch sizes. | | `NREPEAT` | 3 | Default repetitions (with `CHECK`). | | `XREPEAT` | 66 | Default repetitions (without `CHECK`). | | `WARMUP` | 2 | Number of untimed warmup iterations. | | `MAX_KERNEL_DIM` | 80 | Maximum supported matrix dimension. | | `ALIGNMENT` | `LIBXS_ALIGNMENT` | Buffer alignment in bytes. |

LIBSMM Kernel Parameters

The LIBSMM library generates OpenCL kernels at runtime for any requested (M, N, K) shape within `MAX_KERNEL_DIM`. Performance can be improved by supplying tuned parameters, either embedded at build time or loaded at runtime.

Environment Variables (LIBSMM)

Transpose kernel:

| Variable | Default | Description | |||| | `OPENCL_LIBSMM_TRANS_BUILDOPTS` | - | Extra OpenCL build options for transpose kernels. | | `OPENCL_LIBSMM_TRANS_INPLACE` | 0 | Non-zero: in-place transpose (no local memory). | | `OPENCL_LIBSMM_TRANS_BM` | auto | Block size in M-direction (0 < BM <= M). |

Multiply kernel:

| Variable | Default | Description | |||| | `OPENCL_LIBSMM_SMM_BUILDOPTS` | - | Extra OpenCL build options for multiply kernels. | | `OPENCL_LIBSMM_SMM_PARAMS` | auto | Tuned parameters: `0` = disable, or path to CSV file. | | `OPENCL_LIBSMM_SMM_BS` | auto | Intra-kernel mini-batchsize for amortizing atomic updates. | | `OPENCL_LIBSMM_SMM_BM` | auto | Block size in M-direction (0 < BM <= M). | | `OPENCL_LIBSMM_SMM_BN` | auto | Block size in N-direction (0 < BN <= N). | | `OPENCL_LIBSMM_SMM_AP` | auto | Access pattern for the parameter/stack array. | | `OPENCL_LIBSMM_SMM_AA` | auto | Access pattern for the A-matrix array. | | `OPENCL_LIBSMM_SMM_AB` | auto | Access pattern for the B-matrix array. | | `OPENCL_LIBSMM_SMM_AC` | auto | Access pattern for the C-matrix array. |

The full list of tunable parameters and their accepted values is available via `tune_multiply.py --help`.

NOTE: Some parameters produce distinct code paths (e.g., `OPENCL_LIBSMM_SMM_BS=1` vs. `OPENCL_LIBSMM_SMM_BS=2`), so the parameter space is non-smooth.

Tuned Parameters

Pre-tuned parameter sets for various GPUs are stored as CSV files in the `params/` directory:

| File | Device | ||| | `tune_multiply_PVC.csv` | Intel Data Center GPU Max (PVC) | | `tune_multiply_BMG.csv` | Intel Battlemage (BMG) | | `tune_multiply_A100.csv` | NVIDIA A100 | | `tune_multiply_H100.csv` | NVIDIA H100 | | `tune_multiply_GH200.csv` | NVIDIA GH200 | | `tune_multiply_V100.csv` | NVIDIA V100 | | `tune_multiply_P100.csv` | NVIDIA P100 | | `tune_multiply_Mi250.csv` | AMD MI250 |

Parameters are selected by matching the device ID at runtime. When no match is found, the best-matching set is used as fallback. Tuned parameters for single and double precision can coexist in the same CSV and are distinguished automatically.

Loading Parameters Parameters are embedded into the binary at build time (via `tool_opencl.sh`). They can also be overridden at runtime:

```
## Use embedded parameters (default)
./acc_bench.x 5 30000 13 5 7

## Disable tuned parameters (use defaults)
OPENCL_LIBSMM_SMM_PARAMS=0 ./acc_bench.x 5 30000 13 5 7

## Load parameters from a specific CSV file
OPENCL_LIBSMM_SMM_PARAMS=params/tune_multiply_PVC.csv ./acc_bench.x 5 30000 13 5 7
```

To rebuild with a specific device's parameters embedded:

```
make realclean
make WITH_GPU=PVC
```

Auto Tuning

The auto-tuning framework uses OpenTuner to explore the parameter space for each (M, N, K) kernel. The tuner communicates with the benchmark driver solely through environment variables.

```
cd samples/smm
pip install -r requirements.txt
```

```
./tune_multiply.py 13x5x7
```

The script accepts `--stop-after=N` to limit tuning to N seconds. Without a limit, OpenTuner decides when to stop. Tuning can be interrupted with Ctrl-C and results are still written.

Output is a JSON file encoding the benchmark, precision, kernel shape, problem size, and achieved GFLOPS/s, e.g., `tune_multiply-float-12x12x12-s15-60gflops.json`.

NOTE: Delete the `opentuner.db` directory before tuning a different kernel to avoid cross-contamination (`tune_multiply.sh` does this automatically).

If an environment variable for a parameter is already set (e.g., `OPENCL_LIBSMM_SMM_BM`), that parameter is held fixed and excluded from tuning, allowing directed exploration of the remaining parameter space.

Tuning Multiple Kernels The `tune_multiply.sh` wrapper script handles multiple kernels via triplet specifications:

```
./tune_multiply.sh -t 300 -j 8 -i 1  4 10 15, 6 7 8, 23
```

Triplets are comma-separated groups of (M, N, K)-extents. Each group is expanded as a Cartesian product, groups are concatenated, and duplicates are removed. The example above expands to 55 kernels.

| Option | Description | ||| | `-t <seconds>` | Time limit per kernel (default: 160s for updates). | | `-j <parts>` | Total number of partitions for distributed tuning. | | `-i <index>` | Partition index to process (1-based). | | `-c` / `--continue` | Continue after Ctrl-C instead of stopping all kernels. | | `-u` / `--update` | Re-tune existing JSON files (parse M,N,K from filenames). | | `-p <path>` | Directory containing JSON files to update. | | `-a <level>` | Tuning level (default: 1 for updates). |

The script tunes 1266 default kernels. With `-t 300 -j 8`, each partition takes approximately 13 hours.

Bulk Tuning with MPI Multiple devices can be used in parallel by launching `tune_multiply.sh` under MPI (one process per device):

```
MAXTIME=200 NPARTS=8 UPDATE=1 JSONDIR=params/pvc mpirun \
  ./tune_multiply.sh -i 1 : \
  ./tune_multiply.sh -i 2 : \
  ./tune_multiply.sh -i 3 : \
  ./tune_multiply.sh -i 4 : \
  ./tune_multiply.sh -i 5 : \
  ./tune_multiply.sh -i 6 : \
  ./tune_multiply.sh -i 7 : \
  ./tune_multiply.sh -i 8 \
>out.log 2>&1
```

Managing Tuned Parameters JSON files are automatically merged into a CSV file by `tune_multiply.py -m`. The device name in JSON files can be updated to match the current driver version with `tune_multiply.py -u`. Different problem sizes are collapsed to a maximum, so tuning for a non-default problem size requires care to avoid dominance by larger results.

To retune an existing parameter set:

```
make realclean
make WITH_GPU=P100                          # embed current parameters
smm/tune_multiply.sh -p smm/params/p100 -u # retune from embedded baseline
cp tune_multiply.csv smm/params/tune_multiply_P100.csv
```

NOTE: Keep the GPU driver state persistent during tuning to avoid repeated initialization overhead (e.g., `nvidia-smi -pm ENABLED` on headless Nvidia systems).

Files

| File | Description | ||| | `acc_bench.c` | Benchmark driver: batched SMM with H2D, device, and host timing. | | `acc_bench.h` | Stack initialization and matrix setup helpers. | | `acc_libsmm.h` | ACC LIBSMM interface definition (transpose + multiply). | | `opencl_libsmm.c` | OpenCL LIBSMM implementation: kernel generation, parameter handling, dispatch. | | `opencl_libsmm.h` | LIBSMM internal types: kernel configs, tunables, parameter I/O. | | `kernels/multiply.cl` | OpenCL C multiply kernel. | | `kernels/transpose.cl` | OpenCL C transpose kernel. | | `tune_multiply.py` | OpenTuner-based auto-tuning script for multiply kernels. | | `tune_multiply.sh` | Wrapper: triplet expansion, partitioning, bulk/MPI tuning. | | `acc_triplets.sh` | Triplet generation helper. | | `params/*.csv` | Pre-tuned parameter sets per device. |